

가로등 장애물 구현 가이드라인

Godot 4 · 프로그래머용

1. 개요

오브젝트에서 부채꼴 모양의 빛이 나옵니다.

빛의 색상은 흰색 또는 검은색이며, 플레이어와 같은 색일 때만 통과 가능합니다.

다른 색이면 사망 처리됩니다.

색이 전환되는 동안(흰↔검)은 어떤 색의 플레이어도 안전합니다.

2. 기존 코드 연동 방식

player.gd 는 수정하지 않습니다.

기존 `_check_color_death()` 함수가 아래 조건으로 사망 판정합니다.

```
Area2D 가 "death_zones" 그룹에 속해 있고
부모 노드의 color_state != player_color → 사망
```

색 전환 중일 때는 `_area.remove_from_group("death_zones")` 으로 판정 영역을 일시 제거하면, `player.gd` 수정 없이 안전 구간을 구현할 수 있습니다.

전환이 끝나면 다시 `_area.add_to_group("death_zones")` 으로 복구합니다.

3. 씬 구조 (LightObstacle.tscn)

LightObstacle (Node2D)	← light_obstacle.gd 부착
├─ LampSprite (Sprite2D)	← 가로등 오브젝트 이미지
├─ LightVisual (Sprite2D)	← 부채꼴 빛 이미지 (반투명)
└─ DetectionArea (Area2D)	← 사망 판정 영역
└─ CollisionPolygon2D	← 코드에서 부채꼴로 자동 생성

⚠ **DetectionArea** 설정: **Monitoring = ON**, Monitorable = OFF

4. 스크립트 (light_obstacle.gd)

```
extends Node2D

# — 인스펙터 —
@export_enum("BLACK:0", "WHITE:1")
var start_color: int = ColorDefs.BLACK
```

```

@export var hold_time: float      = 2.0    # 한 색이 유지되는 시간(초)
@export var transition_time: float = 0.5    # 색 전환에 걸리는 시간(초)
@export var light_radius: float   = 200.0  # 부채꼴 반지름(px)
@export var light_angle: float    = 90.0   # 부채꼴 각도(도)

# — player.gd 가 읽는 값 —————
# _check_color_death() 의 plat.get("color_state") 로 접근됨
var color_state: int = ColorDefs.BLACK

# — 내부 —————

var _is_transitioning: bool = false
var _timer: float = 0.0

@onready var _area: Area2D = $DetectionArea
@onready var _polygon: CollisionPolygon2D = $DetectionArea/CollisionPolygon2D
@onready var _visual: Sprite2D = $LightVisual

func _ready() -> void:
    color_state = start_color
    _timer = hold_time
    _area.add_to_group("death_zones")
    _build_fan_polygon()
    _update_visual()

func _process(delta: float) -> void:
    _timer -= delta
    if _timer <= 0.0:
        if _is_transitioning:
            _end_transition()
        else:
            _start_transition()

# 전환 시작: death_zones 에서 제거 → 플레이어 안전
func _start_transition() -> void:
    _is_transitioning = true
    _timer = transition_time
    _area.remove_from_group("death_zones")
    # TODO: 시각 전환 연출 (트윈, 깜빡임 등)

# 전환 완료: 색 교체 후 death_zones 복구
func _end_transition() -> void:
    _is_transitioning = false
    color_state = ColorDefs.WHITE if color_state == ColorDefs.BLACK \
        else ColorDefs.BLACK

    _timer = hold_time
    _area.add_to_group("death_zones")
    _update_visual()

# 부채꼴 충돌체 자동 생성
# 노드 회전값으로 방향 조절 가능 (에디터에서 LightObstacle 회전)
func _build_fan_polygon() -> void:
    var points := PackedVector2Array()
    points.append(Vector2.ZERO) # 부채꼴 꼭짓점(중심)

```

```

var half_rad := deg_to_rad(light_angle * 0.5)
var steps    := 16 # 높을수록 곡선이 부드러움

for i in steps + 1:
    var angle := -half_rad + (half_rad * 2.0 / steps) * i
    points.append(Vector2(cos(angle), sin(angle)) * light_radius)

_polygon.polygon = points

# 비주얼 색상 업데이트
func _update_visual() -> void:
    if _visual:
        _visual.modulate = Color.BLACK if color_state == ColorDefs.BLACK \
            else Color.WHITE

```

5. 인스펙터 설정 항목

항목	설명	기본값
Start Color	시작 색 (BLACK / WHITE)	BLACK
Hold Time	한 색이 유지되는 시간 (초)	2.0
Transition Time	색 전환에 걸리는 시간 (초)	0.5
Light Radius	부채꼴 반지름 (px)	200.0
Light Angle	부채꼴 각도 (도)	90.0

6. 방향 조절

부채꼴은 항상 **+X 방향(오른쪽)** 기준으로 생성됩니다.

에디터에서 **LightObstacle** 노드를 **회전**시키면 빛의 방향을 자유롭게 설정할 수 있습니다.

7. 주의사항

- **player.gd** 는 수정하지 않습니다. 기존 **death_zones** 그룹 시스템을 그대로 활용합니다.
- **DetectionArea** 의 **Monitoring** 은 반드시 **ON** 으로 설정해야 합니다.
- **LightVisual** 이미지는 부채꼴 모양의 반투명 스프라이트를 사용하세요.
- 시각적 전환 연출(**_start_transition** 내 TODO)은 별도로 구현이 필요합니다.
- **light_radius / light_angle** 변경 시 **_build_fan_polygon()** 은 **_ready()** 에서만 호출됩니다. 런타임 중 변경이 필요하면 직접 재호출하세요.